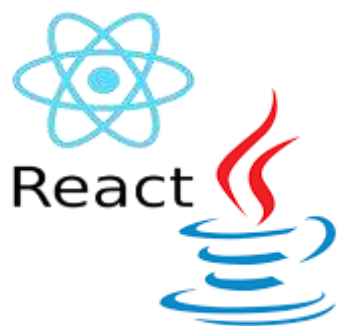


Java Full Stack Development



A **Full Stack Developer** is a versatile professional who possesses the skills and knowledge to work on both the frontend (user interface) and backend (server-side logic and database) of a web application. This comprehensive role requires a deep understanding of how these different layers interact to deliver a complete, functional user experience.



Introduction to Full Stack Development

A Java Full-Stack Developer role involves designing, developing, and maintaining entire web applications, covering both the client-side (frontend) and server-side (backend) components. Responsibilities include building user interfaces with HTML, CSS, and JavaScript frameworks like React or Angular, developing server-side logic using Java and frameworks like Spring, working with databases, creating REST APIs, and overseeing the complete development lifecycle from analysis to deployment



Key Responsibilities

- Designing intuitive user interfaces. Writing
- robust business logic.
- Managing and optimizing databases.

Industry Demand

Full stack developers are in high demand across the IT industry, as they can oversee an entire project from conception to deployment, making them invaluable assets to any development team.

What You Will Achieve

This course is meticulously designed to transform aspiring developers into proficient Java full stack professionals. By the end, you'll have a robust skill set applicable to real-world projects.



Java Backend Mastery

Master Java for robust backend development, including advanced concepts and efficient coding practices.



Database Management

Work confidently with both relational (MySQL)



Frontend Expertise

Gain proficiency in core frontend technologies like HTML, CSS, JavaScript, and leading frameworks such as React.



API Development

Build powerful RESTful APIs and secure web services to facilitate seamless communication.



Spring Boot Hands-On

Acquire practical experience with the industry-standard Spring Boot framework for rapid application development.

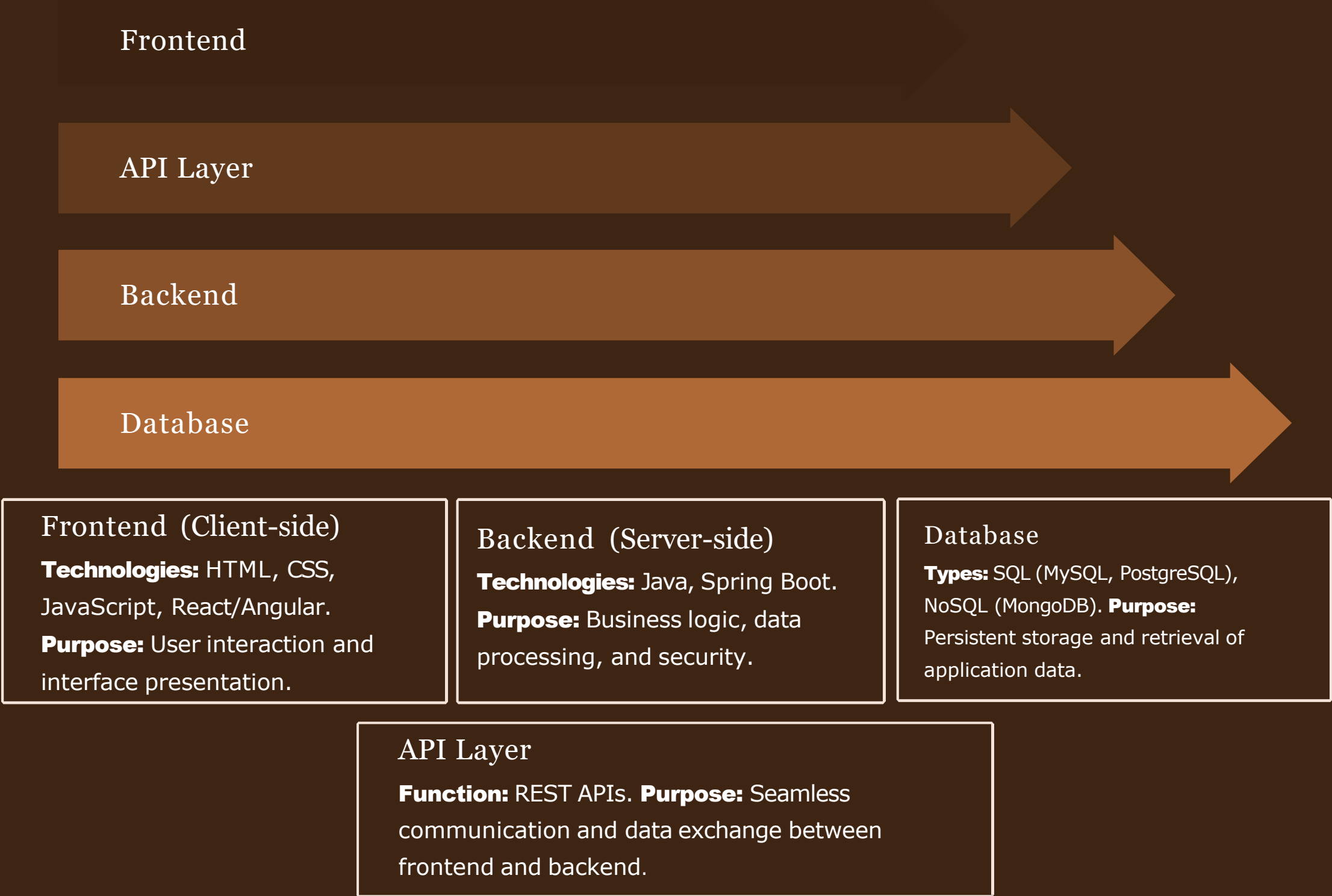


Cloud Deployment

Learn to deploy your applications to cloud platforms, ensuring scalability and accessibility.

Full Stack Architecture Explained

Understanding the layered architecture is crucial for a full stack developer. This diagram illustrates how different components integrate to form a complete web application.

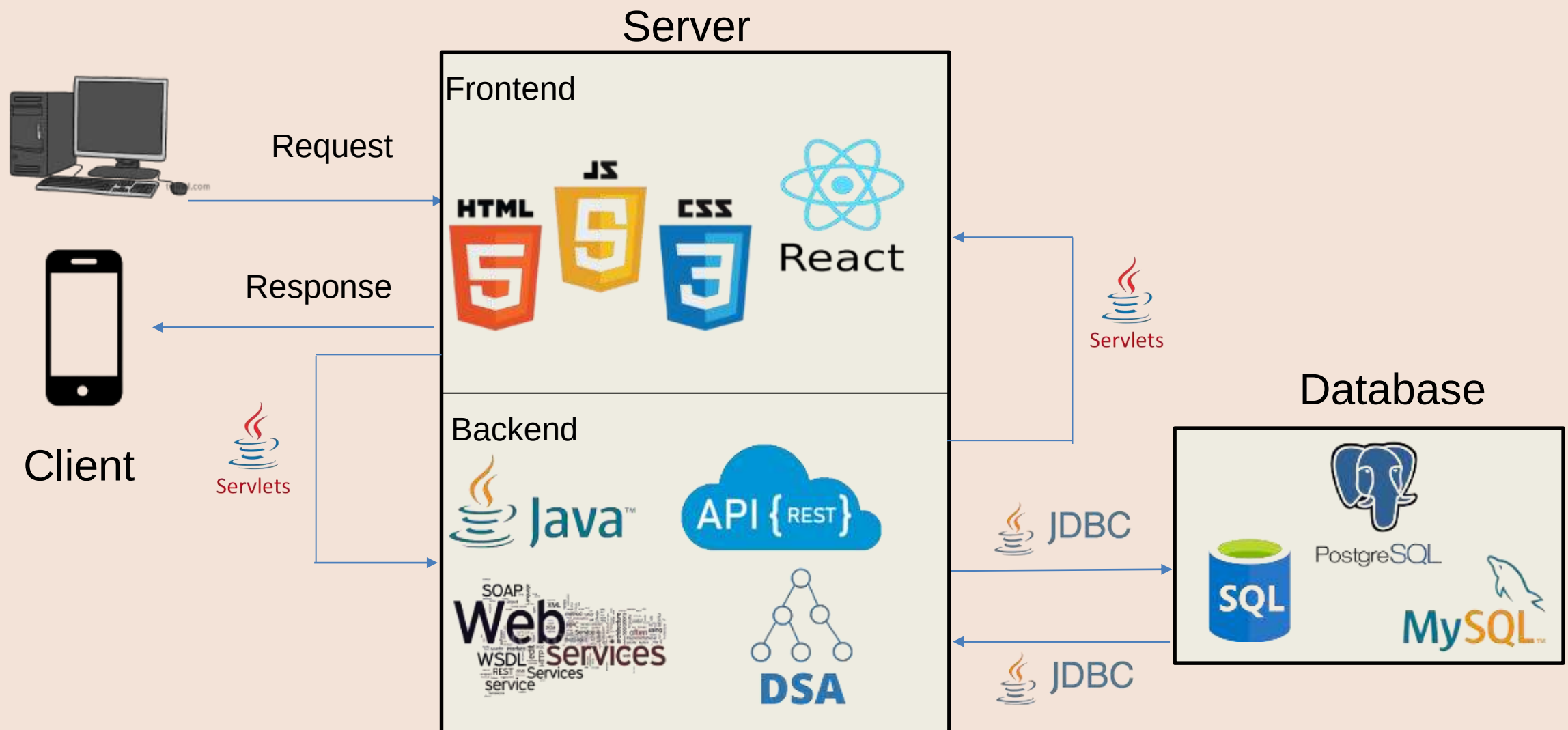


Java Full Stack Architecture

Client Server & Web Application

Client – The device or software application that initiates a request for a service or resource, such as a web browser, mobile app, or desktop application .

Server – It is a powerful computer or software system that centralizes and provides shared resources, data, and services to other computers, known as clients, over a network.



Deep Dive into Frontend Development

Frontend development focuses on everything a user sees and interacts with directly in their web browser. It's about creating responsive, intuitive, and visually appealing interfaces.

Core Technologies

HTML5: For structuring web content. **CSS3:** For styling and visual presentation.

JavaScript ES6+: For interactive behavior and dynamic content

Modern Frameworks

Master leading JavaScript frameworks that simplify complex UI development: **Angular** (with its strong emphasis on data binding, components, and services) or **React** (known for its component-based architecture, hooks, and efficient state management).



"The frontend is where user experience truly comes alive; it's the art of making complex functionalities feel simple and intuitive."

Asynchronous Communication

Understand how to make efficient data requests to the backend without reloading the page using **AJAX** and the **Fetch API**.

Backend Development with Java



The backend is the engine of any application, handling data, logic, and server operations. Java, combined with Spring Boot, provides a powerful and scalable foundation for building enterprise-grade applications.

Core Java Fundamentals

Solidify your understanding of Object-Oriented Programming (OOP) concepts, collections, and robust exception handling.

Advanced Java Web

Explore traditional Java web technologies like Servlets and JSP to grasp the foundational concepts of web interactions.

Spring Boot Framework

Unlock rapid development capabilities with Spring Boot for creating REST APIs, managing security, and handling authentication.

Data Persistence with ORM

Learn to interact with databases efficiently using Hibernate and JPA (Java Persistence API), simplifying data mapping and operations.

Database Management

Effective data management is critical for any application. This section equips you with the skills to work with both traditional relational and modern non-relational databases.



SQL Databases

- Focus on **MySQL** and **PostgreSQL**. Master tables, joins, and transactions.
- Learn to write optimized queries for performance.

NoSQL Databases

- Explore **MongoDB** for document-based storage.
- Understand its scalable and flexible nature.

Database Integration

- Seamlessly connect and interact with databases using Spring Boot.
- Implement efficient data access layers for your applications.

Tech Stack

Database



- | | |
|---|---|
| <ul style="list-style-type: none"> • DATA • DATABASE • DBMS • RDBMS • SQL RULES OF E.F.CODD • DATA TYPES • CONSTRAINTS • ATTRIBUTES | <ul style="list-style-type: none"> • STATEMENTS OF SQL • DATA DEFINITION LANGUAGE (DDL) • DATA MANIPULATION LANGUAGE (DML) • TRANSACTION CONTROL LANGUAGE (TCL) • DATA CONTROL LANGUAGE (DCL) • DATA QUERY LANGUAGE • OPERATORS • FUNCTIONS • GROUP BY CLAUSE • HAVING CLAUSE |
|---|---|

- | | | |
|---|---|---|
| <ul style="list-style-type: none"> • ORDER BY CLAUSE • SUBQUERY • NESTED SUB QUERY • SINGLE ROW SUB QUERY • MULTI ROW SUB QUERY • EMP-MGR RELATIONSHIP • JOINS • CARTESIAN / CROSS JOIN | <ul style="list-style-type: none"> • INNER / EQUI JOIN • OUTER JOIN • LEFT OUTER JOIN • RIGHT OUTER JOIN • FULL OUTER JOIN • SELF JOIN • NATURAL JOIN • PSEUDO COLUMNS • ROWNUM • ROWID | <ul style="list-style-type: none"> • CO - RELATED SUB QUERY • SET THEORY • UNION • UNION ALL • INTERSECT • WINDOW FUNCTIONS • FUNCTIONAL DEPENDENCY • NORMALIZATION • ACID PROPERTY • BASIC PL SQL CONTENTS |
|---|---|---|

- | | |
|--|---|
| <ul style="list-style-type: none"> • 1NF • 2NF • 3NF • BCNF • ACID PROPERTY • BASIC MYSQL WORKBENCH COMMANDS | <ul style="list-style-type: none"> • VIEW • PROCEDURE • FUNCTION • TRIGGER • INDEX |
|--|---|

Level 1- Pre- Programming

- Introduction to Programming Language
- Language
- Programming Language and it's type and paradigms.
- Levels of Language
- Java as backend Programming Language
- History of Java
- Versions and Flavors (Editions) of Java
- Role of Java developer
- Features of Java
- Source code
- Java's Magic Bytecode
- The Java Environment
- JDK
- JRE
- JVM architecture
- Basic Language Elements
- Tokens
- Keywords
- Identifiers
- Comments
- Separators
- Literals
- Datatype
- Variables

- Operators
- Assignment operator
- Arithmetic operator
- Compound Assignment operator
- Comparison operator
- Logical operator
- Bitwise operator
- Conditional operator – The Ternary Operator
- Increment and Decrement operator
- Miscellaneous operator
- Internal working of Printing statement
- Java's Boiler Plate – Program structure
- Class
- Main methods
- Compilation
- Execution

- The Control Flow Statements
- Decision Making Statement
- Simple if
- If else
- If else if (ladder)
- Switch
- Looping Statement
- For loop
- While loop
- Do-while loop
- For each loop - JDK 5
- Branching Statement
- Return
- Break
- Continue



Level 2- Core Programming

- | | | |
|--|---|---|
| <ul style="list-style-type: none"> • Object Oriented Programming • Public class • Multiple classes • Static Members • Static method • Static variable • Static Initializers • Class Loading Process for Static Members | <ul style="list-style-type: none"> • Class Area (Meta Space & Virtual Method Table) • Java program without main() • Object & Object reference • Creating and operating Objects • Ways to create object • Class Fundamentals • Access Control • Inner Class • Anonymous Classes | <ul style="list-style-type: none"> • Use of modifiers with classes and methods • Non-static Members • Non-static Method • Non-static Variable • Non-static Initializers • Object creation & Loading process |
|--|---|---|

- | | | |
|---|--|---|
| <ul style="list-style-type: none"> • The Constructors • Types of Constructor • Default Constructor • No-argument Constructor • Parameterized Constructor • Copy Constructor • Singleton Class – Factory Design Pattern • Use of super keyword • Constructor chaining | <ul style="list-style-type: none"> • Introduction to OOP's • Advantages & Disadvantage of OOP • Principal Pillar's • Encapsulation • Private modifier • Data hiding • Relationship • Inheritance • Types of Inheritance • Type Casting | <ul style="list-style-type: none"> • Polymorphism • Abstraction • Abstract keyword • Abstract class and its characteristics • Abstract method and overriding • Abstract class vs Concrete class |
|---|--|---|

- | | | |
|--|--|--|
| <ul style="list-style-type: none"> • Introduction to Interface • Functionality of Interface • Marker Interface • Functional Interface • Extending & Implementing Interface • The Object Class • Why object class is the super class • Methods of Object class and it's usage | <ul style="list-style-type: none"> • Introduction to OOP's • Advantages & Disadvantage of OOP • Principal Pillar's • Encapsulation • Private modifier • Data hiding • Relationship • Inheritance • Types of Inheritance • Type Casting | <ul style="list-style-type: none"> • Cloning an object • Overriding Object class methods • Introduction to Exception • Types of Errors • Java Exception Hierarchy • Exception Classes in Java • Common Exception Classes • Handling Exceptions • Auto Closeable interface • Throwing Exception |
|--|--|--|

- Arrays
- What is an array?
- Features and limitations
- Memory representation of arrays
- Array vs variable
- Types of Array
- Declaration and Initialization
- Array Traversal
- Advanced Topics
- String
- Introduction
- What is a String in Java?
- Immutable nature of String
- String pool and memory allocation
- String literal vs new String()
- Creating Strings
- Concatenation using + and concat()
- Methods
- String Comparison
- Reference comparison (==)
- Content comparison (equals())
- Lexicographical comparison (compareTo())
- String Buffer
- Content comparison (equals())
- Creating a StringBuffer

- Common Methods
- StringBuilder
- Regular Expression
- What is a regular expression?
- Use cases (pattern matching, validation, etc.)
- Regex Classes in Java
- Regex Syntax
- Regex Methods
- Wrapper Class
- Introduction to Wrapper Class
- What are Wrapper Classes?
- Why do we need wrapper classes?
- Working with Collections (e.g., ArrayList)
- Type conversion
- Utility methods
- Autoboxing and Unboxing
- Automatic conversion between primitive and wrapper types

- Collection Framework
- Introduction to Collection
- What is a Collection?
- Need for Collection Framework
- Arrays vs Collections
- Overview of java.util package
- Collection hierarchy and architecture
- Core Interface of Collection Framework
- Collection interface (base for most)
- List
- Set
- Queue
- Map
- Collections class
- Arrays class
- Generics in Collections
- Iteration and Traversal
- Comparable vs Comparator
- Multithreading
- What is a thread?
- Process vs Thread
- Creating Threads
- Thread Methods
- Thread priorities and behavior
- Synchronization
- Inter-thread Communication

GUI {Graphical User Interface}



- | | | |
|---|--|---|
| <ul style="list-style-type: none"> • Basic HTML • What is HTML? • Document Structure • Headings • Paragraphs • Line breaks and horizontal rules • Comments • Text Formatting Tags | <ul style="list-style-type: none"> • Bold, Italic, Underline • Superscript and Subscript • Quotes & Citations • HTML Elements • Div () & Span () • Lists • Ordered • Unordered • Definition | <ul style="list-style-type: none"> • Links and Images • Anchor • Image • Image Maps • Download & External Links • Forms • Media Elements{Audio and Video} • Semantic Tags • Inputs |
|---|--|---|



- | | | |
|--|--|---|
| <ul style="list-style-type: none"> • Basics • What is CSS? (Inline, Internal, External) • Syntax (Selector, Property, Value) • Comments • Selectors • Basic Selectors • Attribute Selectors • Colors and Units | <ul style="list-style-type: none"> • Text and Fonts • Box Model margin, border, padding, width, height box-sizing • Positioning{static, relative} • Display and Visibility • Flexbox • Grid | <ul style="list-style-type: none"> • Backgrounds • Borders and Shadows • Transitions and Transforms • Animations • keyframes |
|--|--|---|



- | | | |
|---|---|---|
| <ul style="list-style-type: none"> • Beginner Level – Foundations • Basics { Introduction , Datatypes etc.} • Control Structures • Functions • Arrays and Objects • String Methods • Destructuring | <ul style="list-style-type: none"> • Spread and Rest • Error Handling • DOM Manipulation (Browser) • Asynchronous JavaScript • Callbacks • Promises • async/await • Error handling with async/await | <ul style="list-style-type: none"> • Closures and Lexical Environment • this Keyword • Classes and Prototypes • Module System • JavaScript in the Browser • Tooling (Intro) |
|---|---|---|



- Introduction
- How React Works
- JSX (JavaScript XML)
- Components
- State
- Props
- Synthetic Events
- Conditional Rendering
- useRef Hook
- Context API
- useEffect Hook
- Form Handling
- useMemo Hook
- useCallback Hook
- Error Boundaries
- React Router DOM
- Redux / Redux Toolkit
- Axios
- Data API
- Styling in React
- Authentication & Authorization



- Introduction
- Git, GitHub, GitHub Desktop, Git Bash
- Install & first-time setup
- Configure Git
- Authenticate
- Repository.
- Clone your repo
- Make a change and commit
- Push to GitHub
- Branching & PR
- Git Bash
- Commands
- Branching + Pull Request
- Pull latest changes





- JDBC
- JDBC Architecture
- Types of JDBC Drivers
- Steps of JDBC
- Implementing the Steps of JDBC in program
- Statement and Prepared Statement
- Result Set for Fetching Data From Database
- Batch Processing
- Callable Statement
- Stored Procedures
- JDBC Connection Pooling
- Project that illustrates the usage of JDBC with PostgreSQL databases



- Introduction to Servlets
- Working of Servlet
- Program to create Servlet using GenericServlet and HttpServlet
- How to read data from HTML form
- Servlet Life Cycle
- Welcome File List
- Load On Start up
- Request Dispatcher and sendRedirect method
- Servlet Context and Servlet Config
- Session Management
- Cookies and how to create cookie
- Project that illustrates the usage of Servlets with web applications



- Introduction to JSP
- JSP tags
- Action tags
- JSP Implicit Object
- EL
- JSTL
- JSP Directives
- JSP Life Cycle



- Introduction to Hibernate
- Difference between JDBC and Hibernate
- Advantages of Hibernate
- JPA
- ORM
- Persistence.xml file
- Hibernate Relational Mapping
- Hibernate And Database
- Entity class
- Cascading
- Hibernate caching
- HQL
- Hibernate Life Cycle
- CRUD operation on DB using JPA



- Spring core
- Introduction to EJB
- Difference Between EJB and Spring
- Creating Spring Maven
- Spring and Advantages of Spring
- XML-based Bean Config
- Spring Container
- BeanFactory
- Application Context
- Configurable Application Context
- Annotations
- Dependency Injection
- Inversion of Control
- Bean Scope
- Spring Life Cycle



- Introduction to MVC
- MVC Architecture
- Spring Web MVC
- Front Controller
- Program to send data from View to Controller
- Program to send data from Controller to View
- Introduction to View Resolver
- Introduction to Request handling
- Annotations (@Controller, @Request Mapping, @Request Params, @ModelAttribute)
- JSTL (JSP Standard Tag Library)
- Project that follows the Model-View-Controller (MVC) architecture.



- Introduction to Spring Boot
- RESTful web service
- REST API
- JSON and how to create JSON
- Steps to build Spring Boot Project
- Workflow of Spring Boot Project
- Difference Between @Controller and @RestController
- Creating a REST API to Perform CRUD Operations
- Custom Query
- Custom exception
- Handling the Exception
- ResponseEntity
- Spring Boot profiles
- DATA JPA
- PostgreSQL Database
- HTTP Status Codes
- Swagger documentation



- Introduction to Spring Security
- Why security is important
- Creating a Spring Boot Project with Spring Security
- Basic configuration and annotations
- Authentication and Authorization
- How Spring-Boot security works internally
- How to set our own username and password in spring security

- How to assign roles in spring-Boot security
- How to print session Id on the browser
- Setting up OAuth 2.0 login (Google, Facebook, GitHub login integration)
- CSRF (Cross-Site Request Forgery)
- JWT{Json Web Token}
- Project to implement Spring Security



Hands-On Capstone Projects

The culmination of your learning journey will be building practical, real-world applications. These projects simulate industry scenarios, allowing you to apply all acquired skills and build a robust portfolio.



1 NASA Astronomy API Web Application

Create a dynamic web application for displaying APOD for fetching the data from official NASA APIs and implement it in middleware to store data in database to provide user GUI details.

2 Hospital Management System with Spring AI

Build a hospital management system with help of spring AI in order to get the client data and processed from AI to provide user seamless resolution to their health related quires.

3 Internet banking System.

Design and develop a web application with high security measure in order to bank fund transfer from different accounts and providing various facilities of a banking application.

Get Hands-on Experience on Projects to evolve your tech skills

Career Opportunities

The demand for full stack developers continues to soar, making it one of the most rewarding career paths in technology. Your versatile skill set will open doors across various industries.



Versatile Roles

- 1
 - ♦ Full Stack Developer Backend
 - ♦ Engineer Java Developer Software
 - ♦ Engineer

Thriving Industries

- 2
 - ♦ IT Services & Consulting E-commerce &
 - ♦ Retail Tech FinTech & Banking Dynamic
 - ♦ Startups

Why Full Stack Developers Are In Demand:

Efficiency: One developer can manage multiple application layers, streamlining development processes.

Speed: Faster product delivery cycles due to reduced handover and improved communication between teams.

A versatile skill set allows them to quickly adapt to new technologies and project requirements.